CGE-P LAB CURRICULUM · V2

Lab 2.2: The Remote State Backend

Terraform state is the most sensitive artifact in this entire course and the one nobody thinks about. It holds every attribute of every resource you have ever created, in plaintext, including values you marked `sensitive`. By default it sits on your laptop. This lab moves it somewhere defensible, and in doing so unblocks something you would not otherwise discover is broken until Chapter 7.

For reading. Code lines wrap to fit the page; copy code from the repository, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from the repository (`GRCEngClub/cgep-labs`), not from the PDF.

Why this lab exists

Chapter 4 builds a pipeline that runs `terraform init` on a fresh GitHub Actions runner. The capstone requires that pipeline to run `terraform apply` on merge to `main`.

A fresh runner has no state. With a local backend, `terraform plan` on that runner reports every resource as "to be created," because as far as it knows, nothing exists. The first apply after a merge either collides with the resources you already made (`BucketAlreadyExists`) or quietly builds a second copy of your infrastructure.

There is no way to complete the capstone's Layer 3 without this. Build it now, once, and every later lab inherits it.

Learning objectives

- Explain why Terraform state is a confidentiality problem, not just a bookkeeping file.
- Stand up an S3 backend with native locking, KMS encryption, and versioning.
- Solve the bootstrap paradox: creating the bucket that will store the state that describes the bucket.
- Migrate an existing local state into the backend without losing it.

Prerequisites

- AWS account with permissions to create S3 buckets, a KMS key, and a DynamoDB table in `us-east-1`.
- Terraform `>= 1.10`. Native S3 state locking (`use_lockfile`) landed in 1.10; the DynamoDB path below is the fallback for older versions.
- AWS CLI v2, with the `credential_process` profile from Lab 0.1 and `export AWS_PROFILE=cgep` set in this shell.
- 20 minutes.

Estimated time & cost

- Time: 20 minutes.
- Cost: under $0.01 for the bucket. **$1/month for the KMS key**, billed whether or not you use it, and it keeps billing through the deletion waiting period after you schedule it for deletion. The optional DynamoDB table is on-demand billing and costs effectively nothing at lab volume.

Unlike the other labs, **do not destroy this one same-day.** Every subsequent lab stores its state here. Destroy it at the end of the course, after Lab 6.1.

Concept: what is actually in a state file

Run this against any workspace you have already applied:

```
terraform state pull | jq '.resources[].instances[].attributes | keys' | head -40
```

Every attribute. Not a summary, not a hash: the values. For an RDS instance that includes `password`. For an IAM access key it includes `secret`. For a Lambda it includes the environment variables.

Three consequences worth stating in your own words before you continue:

1. `sensitive = true` **is a display control, not a security control.** It redacts CLI output. The value sits in state in plaintext regardless.
2. **Read access to state is equivalent to read access to your secrets.** Treat the state bucket as a secrets store, because that is what it is.
3. **Write access to state is equivalent to control of your infrastructure.** An attacker who can edit state can make Terraform delete resources it no longer believes it owns, or adopt resources it never created.

That is why this bucket gets the full baseline plus a deliberately narrow bucket policy.

Architecture

```
bootstrap/  (local state, committed, run once)
     |
     | creates
     v
+-----------------------------------------+
|  S3 bucket: <project>-tfstate-<suffix>    |
|    SSE-KMS (CMK, rotation on)   SC-28     |
|    versioning ON                CP-9      |
|    public access block x4       AC-3      |
|    deny non-TLS                 SC-8      |
|    lifecycle: 90d noncurrent    AU-11     |
+-----------------------------------------+
     ^
     | backend "s3" { use_lockfile = true }
     |
every other lab in this course
```

The bootstrap workspace keeps local state forever. That is not a mistake; see Step 4.

Step-by-step walkthrough

Step 1 The bootstrap paradox

You need a bucket to store state. Creating that bucket produces state. Where does *that* state go?

Three real answers:

Approach	How it works	Trade-off
Local bootstrap state, committed	A tiny separate workspace with a local backend creates the bucket. Its state file is committed to git.	The bootstrap state describes only a bucket, a key, and a table. Nothing secret. This is what we do.
Bootstrap, then migrate its own state in	Create the bucket, then point the bootstrap workspace at itself.	Elegant, and it makes the bucket undeletable by Terraform without a dance. Common in production.
ClickOps the bucket once	Create it by hand, never manage it in Terraform.	Honest, and it means your most security-critical bucket is the one resource with no code review. Reject this.

We take the first. The bootstrap state contains no secrets, and keeping it local and committed means a new learner clones the repo and can see exactly what was created.

Step 2 Write the bootstrap workspace

```
mkdir -p terraform/bootstrap && cd terraform/bootstrap
touch main.tf variables.tf outputs.tf
```

```
# main.tf
terraform {
  required_version = ">= 1.10"
  required_providers {
    aws    = { source = "hashicorp/aws", version = "~> 5.0" }
    random = { source = "hashicorp/random", version = "~> 3.6" }
  }
  # No backend block. This workspace is deliberately local.
}

provider "aws" {
  region = var.aws_region

  # CM-6 / CM-8: required tags on every taggable resource.
  default_tags {
    tags = {
      Project         = var.project_name
      Environment     = "shared"
```

```
      ManagedBy       = "terraform"
      ComplianceScope = "cge-p-lab"
    }
  }
}

data "aws_caller_identity" "current" {}
data "aws_partition" "current" {}

resource "random_id" "suffix" {
  byte_length = 4
}

locals {
  account_id  = data.aws_caller_identity.current.account_id
  partition   = data.aws_partition.current.partition
  bucket_name = "${var.project_name}-tfstate-${random_id.suffix.hex}"
}

# ------------------------------------------------------------------------------
# SC-12 / SC-13: customer-managed key, rotation enabled.
# State is a secrets store. It gets a key you control, not an AWS-managed one.
# ------------------------------------------------------------------------------
resource "aws_kms_key" "tfstate" {
  description             = "CMK for Terraform state at rest"
  enable_key_rotation     = true
  deletion_window_in_days = var.kms_deletion_window_days
  policy                  = data.aws_iam_policy_document.kms.json
}

resource "aws_kms_alias" "tfstate" {
  name          = "alias/${var.project_name}-tfstate"
  target_key_id = aws_kms_key.tfstate.key_id
}

data "aws_iam_policy_document" "kms" {
  # Without a root-admin statement the key becomes unmanageable. AWS requires
  # this; omitting it is the single most common way to brick a CMK.
  statement {
    sid       = "EnableAccountRootAdmin"
    effect    = "Allow"
    actions   = ["kms:*"]
    resources = ["*"]
    principals {
      type        = "AWS"
      identifiers = ["arn:${local.partition}:iam::${local.account_id}:root"]
    }
  }
}

resource "aws_s3_bucket" "tfstate" {
  bucket = local.bucket_name
}

# AC-3 / AC-6: ACLs disabled entirely. AWS default since April 2023, and an
# ACL-enabled bucket is itself a Security Hub finding (S3.12).
```

```
resource "aws_s3_bucket_ownership_controls" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  rule {
    object_ownership = "BucketOwnerEnforced"
  }
}

# SC-28: encryption at rest with our CMK. bucket_key_enabled cuts KMS request
# charges by up to 99%; state files are written on every apply, so this matters.
resource "aws_s3_bucket_server_side_encryption_configuration" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id = aws_kms_key.tfstate.arn
    }
    bucket_key_enabled = true
  }
}

# CP-9 / SI-7: versioning is mandatory here, not optional. A corrupted or
# truncated state file is recoverable only if the prior version survived.
resource "aws_s3_bucket_versioning" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  versioning_configuration {
    status = "Enabled"
  }
}

# AC-3: all four flags. Two axes (ACL vs policy) by two states (new vs existing).
resource "aws_s3_bucket_public_access_block" "tfstate" {
  bucket                  = aws_s3_bucket.tfstate.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

# AU-11: keep old state versions long enough to recover, not forever.
resource "aws_s3_bucket_lifecycle_configuration" "tfstate" {
  bucket     = aws_s3_bucket.tfstate.id
  depends_on = [aws_s3_bucket_versioning.tfstate]

  rule {
    id     = "expire-noncurrent-state"
    status = "Enabled"

    # An empty filter applies the rule to every object. Provider 5.x errors
    # if a rule has neither filter nor prefix.
    filter {}

    noncurrent_version_expiration {
      noncurrent_days = var.state_version_retention_days
    }

    abort_incomplete_multipart_upload {
```

```
      days_after_initiation = 7
    }
  }
}

# SC-8: refuse any request that did not arrive over TLS.
data "aws_iam_policy_document" "tfstate" {
  statement {
    sid       = "DenyInsecureTransport"
    effect    = "Deny"
    actions   = ["s3:*"]
    resources = [aws_s3_bucket.tfstate.arn, "${aws_s3_bucket.tfstate.arn}/*"]

    principals {
      type        = "*"
      identifiers = ["*"]
    }

    condition {
      test     = "Bool"
      variable = "aws:SecureTransport"
      values   = ["false"]
    }
  }
}

resource "aws_s3_bucket_policy" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  policy = data.aws_iam_policy_document.tfstate.json
}
```

```
# variables.tf
variable "project_name" {
  type        = string
  description = "Short project identifier. Becomes part of the bucket name."
  default     = "cgep-lab"
  validation {
    condition     = can(regex("^[a-z][a-z0-9-]{2,20}$", var.project_name))
    error_message = "project_name must be 3-21 lowercase alphanumerics or hyphens, starting with a
letter."
  }
}

variable "aws_region" {
  type        = string
  description = "Region for the state backend. Every workspace must use the same one."
  default     = "us-east-1"
}

variable "kms_deletion_window_days" {
  type        = number
  description = "Waiting period before a scheduled key deletion completes. AWS allows 7-30."
  default     = 7
  validation {
```

```
    condition     = var.kms_deletion_window_days >= 7 && var.kms_deletion_window_days <= 30
    error_message = "kms_deletion_window_days must be between 7 and 30."
  }
}

variable "state_version_retention_days" {
  type        = number
  description = "How long superseded state versions are kept before expiry."
  default     = 90
}
```

```
# outputs.tf
output "state_bucket" {
  value       = aws_s3_bucket.tfstate.id
  description = "Bucket name. Paste into every other workspace's backend block."
}

output "state_kms_key_arn" {
  value       = aws_kms_key.tfstate.arn
  description = "CMK ARN protecting state at rest."
}

output "backend_block" {
  description = "Copy-paste backend configuration for every other lab."
  value       = <<-EOT
    backend "s3" {
      bucket       = "${aws_s3_bucket.tfstate.id}"
      key          = "CHANGE-ME/terraform.tfstate"
      region       = "${var.aws_region}"
      encrypt      = true
      kms_key_id   = "${aws_kms_key.tfstate.arn}"
      use_lockfile = true
    }
  EOT
}
```

That last output is worth stealing as a pattern. A module that emits its own consumer configuration removes an entire category of transcription error.

Step 3 Apply the bootstrap

```
export AWS_PROFILE=cgep
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply -auto-approve tfplan
```

Expected tail:

```
Apply complete! Resources: 10 added, 0 changed, 0 destroyed.
```

Ten resources: the random suffix, the key, the alias, the bucket, and its six configuration resources (ownership controls, encryption, versioning, public access block, lifecycle, policy).

The lifecycle configuration is the slow one and routinely takes **40 to 60 seconds** while the rest finish in under two. That is S3 settling, not a hang. The KMS key takes about 15 seconds.

Capture the backend block:

```
terraform output -raw backend_block
```

Step 4 Commit the bootstrap state

```
git add -f terraform/bootstrap/terraform.tfstate
git commit -m "Bootstrap: state backend created"
```

The `-f` is required because your `.gitignore` excludes `*.tfstate`, correctly, for every other workspace.

Read this before you object. Committing a state file is normally wrong, and it is right here for exactly one reason: this state describes a bucket, a KMS key, an alias, and five configuration resources. There is no credential, no password, no key material in it. A KMS key's *ARN* is not sensitive; its key material never leaves AWS and never appears in state.

Confirm that for yourself rather than trusting the paragraph:

```
terraform state pull | jq '[.resources[].instances[].attributes | keys] | flatten | unique'
```

If you ever add a resource to this workspace that has a secret attribute, this decision becomes wrong and you must revisit it. Note it in the workspace README so the next person knows the constraint.

Step 5 Native locking, and why the DynamoDB table is optional

Two concurrent applies against one state file corrupt it. Terraform prevents that with a lock.

Terraform 1.10 added `use_lockfile = true`, which uses an S3 conditional write to hold the lock. No second service, nothing extra to pay for, nothing extra to secure. **This is the current recommended approach and what this lab uses.**

The older mechanism is a DynamoDB table with a `LockID` string hash key, referenced by the backend's `dynamodb_table` argument. You need it only if you are pinned below Terraform 1.10. If that is you, add:

```
resource "aws_dynamodb_table" "tflock" {
  name         = "${var.project_name}-tflock"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }

  server_side_encryption {
    enabled = true
  }

  point_in_time_recovery {
    enabled = true
  }
}
```

and `dynamodb_table = "<name>"` in the backend block. Running both mechanisms at once is supported and harmless.

Step 6 Point a workspace at the backend

In any lab workspace, add the backend block inside `terraform { }`, giving each workspace a distinct `key`:

```
terraform {
  required_version = ">= 1.10"

  backend "s3" {
    bucket       = "cgep-lab-tfstate-XXXXXXXX"
    key          = "labs/2-3/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    kms_key_id   = "arn:aws:kms:us-east-1:ACCOUNT:key/KEY-ID"
    use_lockfile = true
  }

  required_providers { /* ... */ }
}
```

The `key` is a path inside the bucket, and it is how workspaces stay isolated. Reuse one `key` across two labs and the second lab will believe it owns the first lab's resources, then offer to destroy them. Use `labs/<lab-number>/terraform.tfstate` throughout.

Step 7 Migrate existing local state

If you already applied Lab 2.3 locally, do not start over:

```
cd terraform/primitives/compliant-s3
# add the backend block first, then:
terraform init -migrate-state
```

Terraform detects the local state, asks to copy it up, and answering `yes` uploads it. Verify before deleting anything local:

```
terraform state list
aws s3 ls "s3://<state-bucket>/labs/2-3/"
```

Keep the local `terraform.tfstate.backup` until you have confirmed a clean `plan` against the remote state. "No changes" is the signal that migration worked.

Step 8 Record the outputs in `cgep.env`

Lab 4.3 needs this bucket and key by name, and both strings contain your account ID and a random suffix. Read them out rather than retyping them:

```
{
  echo "export TF_VAR_state_bucket=$(terraform output -raw state_bucket)"
  echo "export TF_VAR_state_kms_arn=$(terraform output -raw state_kms_key_arn)"
} >> ../../cgep.env
source ../../cgep.env
```

If you have not created `cgep.env`, Lab 0.1 Step 15 sets it up. It is the file you source at the start of every session, and it is gitignored because it names your account.

Verification

```
BUCKET=$(cd terraform/bootstrap && terraform output -raw state_bucket)

# SC-28: KMS encryption with your CMK
aws s3api get-bucket-encryption --bucket "$BUCKET"

# CP-9: versioning
aws s3api get-bucket-versioning --bucket "$BUCKET"

# AC-3: all four flags true
aws s3api get-public-access-block --bucket "$BUCKET"

# SC-8: the TLS deny is present
aws s3api get-bucket-policy --bucket "$BUCKET" \
  --query Policy --output text | jq '.Statement[] | select(.Sid=="DenyInsecureTransport")'
```

Then prove the TLS control actually works, rather than merely existing:

```
aws s3api list-objects-v2 --bucket "$BUCKET" --endpoint-url "http://s3.us-east-1.amazonaws.com"
```

Expected: `AccessDenied`. A control you have not watched deny something is a control you are guessing about.

Portfolio submission checklist

- [ ] `terraform/bootstrap/{main.tf,variables.tf,outputs.tf,README.md}` committed.
- [ ] `terraform/bootstrap/terraform.tfstate` committed, with the README explaining why that is safe here.
- [ ] At least one lab workspace has a `backend "s3"` block with its own `key`.
- [ ] `evidence/lab-2-2/backend-verification.json`, the output of the four verification commands.
- [ ] README notes the region, because the backend and every workspace must agree on it.

Troubleshooting

- **`Credentials were refreshed, but the refreshed credentials are still expired.`** If it appears seconds after `aws login`, it is a transient cache race: the CLI returns your prompt before its own credential cache settles. Re-run the command. If it persists, the shell is holding an expired snapshot from `export-credentials --format env`, which outranks `AWS_PROFILE`: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION`, then re-run.
- **`ExpiredToken` from Terraform mid-lab.** `aws login` sessions are short, on the order of fifteen minutes, and a lab is longer than that. Run `aws login --profile default` and re-run the Terraform command; with the `credential_process` profile there is nothing to re-export, because the provider resolves credentials afresh on every run. With remote state nothing is lost. Short-lived credentials expiring is the feature you chose by not creating an access key.
- **`Error: Failed to get existing workspaces: S3 bucket does not exist.`** The backend block runs before anything else, including provider configuration. The bucket must already exist. That is the entire reason for the bootstrap workspace.
- **`AccessDenied` on `kms:GenerateDataKey` during `init`.** The identity running Terraform needs `kms:Encrypt`, `kms:Decrypt`, and `kms:GenerateDataKey` on the state CMK. The root-admin statement in the key policy covers an admin principal; a scoped CI role needs an explicit grant. Lab 4.3 adds it.
- **`Error acquiring the state lock.`** An interrupted run left a lock. Confirm nobody else is applying, then `terraform force-unlock <lock-id>`. With `use_lockfile`, the lock is an object at `<key>.tflock`; you can see it with `aws s3 ls`.
- **`use_lockfile` unrecognized.** You are below Terraform 1.10. Either upgrade or use the DynamoDB table from Step 5.
- **Backend changed and now `init` fails.** Any edit to the backend block requires `terraform init -reconfigure` (discard the old backend) or `-migrate-state` (copy it across). Terraform will not guess which you meant.

- `BucketAlreadyExists` **on bootstrap.** The `random_id` suffix should prevent it. If you hardcoded a name, change it. Bucket names are globally unique across every AWS account.

Cleanup

Do not clean this up until the course is finished. Everything downstream stores state here.

At the end of the course, after Lab 6.1:

```
# Every other workspace must be destroyed first, or you will strand resources
# with no state describing them.
BUCKET=$(cd terraform/bootstrap && terraform output -raw state_bucket)

aws s3api list-object-versions --bucket "$BUCKET" \
  --query '{Objects: Versions[].{Key:Key,VersionId:VersionId}}' --output json \
  | aws s3api delete-objects --bucket "$BUCKET" \
    --delete file:///dev/stdin || true

cd terraform/bootstrap && terraform destroy -auto-approve
```

The KMS key enters its deletion waiting period rather than disappearing, and **bills at $1/month for the whole window**. Set `kms_deletion_window_days = 7` (the minimum) if that bothers you.

Stranding is the real risk here. If you delete the state bucket while another lab's resources still exist, those resources have no state and Terraform can no longer destroy them. You will be deleting them by hand in the console, which is exactly the failure mode this course exists to argue against.

How this feeds the capstone

- **Ch 4**, your pipeline's `terraform init` finds real state, so `plan` shows an accurate diff instead of proposing to create everything. The OIDC role gets scoped read/write to this bucket and key.
- **Capstone Layer 3**, `Apply` on merge to `main` works, because the runner and your laptop share one state. Without this lab that step cannot be completed.
- **Your write-up**, the state backend is a defensible answer to "where do your secrets live and who can read them," which is a question every assessor asks and most candidates have not considered.

Revision history

v2 (current)

- New lab. Without a remote backend a fresh CI runner plans against empty state, which makes the capstone's apply-on-merge step uncompletable.
- Uses `use_lockfile` for native S3 locking, with the DynamoDB table documented as the pre-1.10 fallback.

- The bootstrap workspace keeps local state deliberately, with the reasoning and a command to verify it holds no secrets.

v1

Did not exist. State was local everywhere.